

RepoSetup

Terminal-First v1 - Complete Implementation Specification

Source-of-truth document for implementation in Cursor

Architecture, CLI behavior, integration registry, safety, testing, CI/CD, and release criteria

Product statement RepoSetup is an open-source terminal tool that composes, validates, installs, and diagnoses development stacks through a curated integration registry. The terminal product is v1. A website is explicitly deferred to a future phase.

Document field	Value
Implementation language	TypeScript
Runtime	Node.js
CLI framework	Commander.js
Interactive prompts	@inquirer/prompts
Validation	Zod
Process execution	execa
Tests	Vitest
Workspace	pnpm workspaces
Initial release target	v1.0 - terminal only
Website	Out of scope until post-v1

Prepared for implementation planning. Exact third-party installation commands and version ranges must be verified against current official documentation before each integration is released.

Table of Contents

1. Executive Summary
2. Product Goals and Non-Goals
3. v1 Scope and Supported Integrations
4. User Experience and CLI Principles
5. CLI Command Specification
6. Configuration and Local Metadata
7. System Architecture
8. Core Domain Model
9. Integration Registry
10. Compatibility and Dependency Resolver
11. Installation Plan and Operations
12. Package Manager and Runtime Adapters
13. Project Creation
14. Existing Project Detection
15. Add Integration Flow
16. Remove and Rollback Policy
17. Search, Info, Stack, Doctor, Export and Import
18. Presets
19. Filesystem and Configuration Modification
20. Environment Variables and Secrets
21. Security and Safe Command Execution
22. Error Model and Recovery
23. Logging and Diagnostics
24. Testing Strategy
25. Golden Stack Certification
26. CI/CD and Release Engineering
27. Open-Source Contribution Model
28. Documentation Requirements
29. Implementation Roadmap
30. Definition of Done
- Appendix A. v1 Integration Registry
- Appendix B. Example Configurations
- Appendix C. Cursor Implementation Instructions
- Appendix D. Future Website Boundary

1. Executive Summary

RepoSetup v1 is a terminal-first open-source developer tool. It should reduce the friction of assembling a project from multiple frameworks, libraries, databases, ORMs, testing tools, and development utilities. Instead of acting as a package manager replacement, RepoSetup coordinates existing official tools and package managers through deterministic integration definitions.

The core value is not "install package X." The core value is understanding how technologies fit together: what is required, what conflicts, what configuration files must change, what order operations must run in, and how to verify that the resulting project is healthy.

1.1 Core user journey

```
$ reposetup create my-app

? Framework      Next.js
? Language       TypeScript
? Package manager pnpm
? Styling        Tailwind CSS
? Database       PostgreSQL
? ORM            Prisma
? Validation     Zod
? Unit tests     Vitest
? E2E tests      Playwright

RepoSetup will perform 14 operations.
Review plan? Yes

[create] project
[install] prisma @prisma/client zod
[configure] prisma/schema.prisma
[configure] .env.example
[verify] dependency graph
[verify] project build

Continue? Yes
```

1.2 Architectural rule

Source of truth The integration registry describes technologies. The resolver validates relationships. The planner produces declarative operations. Only the executor is allowed to modify the machine or project.



1.3 v1 safety boundary

- **RepoSetup installs project-level dependencies.** It may use npm, pnpm, bun, uv, and pip through adapters.
- **RepoSetup does not silently install system runtimes.** Node.js, Python, Git, Docker, and other machine-level prerequisites are detected and diagnosed in v1.
- **RepoSetup never requires root/admin privileges for normal project setup.** If a system-level dependency is missing, doctor output explains how to install it using official guidance.

- **RepoSetup never executes commands downloaded from presets.** Presets contain declarative integration IDs and options only.
- **RepoSetup supports dry-run before execution.** Every mutating flow must be previewable.

2. Product Goals and Non-Goals

2.1 Goals

- Create new projects using curated, known-good development stacks.
- Add supported integrations to existing projects when safe.
- Detect frameworks, package managers, dependencies, and key configuration in existing projects.
- Explain compatibility, requirements, recommendations, and conflicts before modifying files.
- Provide searchable terminal discovery for supported technologies.
- Generate reproducible configuration files that can be committed to version control.
- Diagnose broken or incomplete setups with a doctor command.
- Make integration contributions straightforward for open-source contributors.
- Support JavaScript/TypeScript and Python workflows in the first stable release.
- Keep the core deterministic and testable without AI or a remote service.

2.2 Explicit non-goals for v1

- No website or graphical dashboard.
- No Firebase/backend service.
- No accounts, hosted presets, likes, comments, or social/community features.
- No arbitrary npm/PyPI package installation through the registry; only curated integrations.
- No remote executable plugin marketplace.
- No AI-generated shell commands.
- No automatic system runtime installation or sudo/admin elevation.
- No support for every programming language or framework.
- No guarantee that unsupported combinations will work merely because the underlying packages can technically coexist.
- No complex rollback of third-party generators in v1; rollback is conservative and metadata-driven.

2.3 Product principles

Principle	Meaning
Correct before broad	20 reliable integrations are better than 150 unreliable integrations.
Deterministic	Same config and compatible tool versions should resolve to the same plan.
Transparent	Users can inspect every planned operation before execution.
Local-first	Core features do not require an account, server, or network except when dependencies themselves must be downloaded.
Safe by default	No arbitrary shell interpolation, no unknown remote commands, no silent overwrites.
Composable	Integrations declare requirements and operations rather than hardcoding workflow in CLI UI.
Tested combinations	Golden stacks receive real installation tests in CI.

3. v1 Scope and Supported Integrations

v1 should support enough technologies to build useful React/Next.js projects, Node APIs, and Python APIs. Support means more than displaying an integration name: the integration must have verified installation behavior, compatibility metadata, detection logic, a dry-run plan, tests, and documentation.

3.1 v1 integration set

Category	ID	Integration	v1 behavior
Runtime prerequisite	node	Node.js	Detect + verify only
Runtime prerequisite	python	Python	Detect + verify only
JS package manager	npm	npm	Supported
JS package manager	pnpm	pnpm	Preferred JS workflow
JS package manager	bun	Bun package manager	Supported where verified
Python package manager	uv	uv	Preferred Python workflow
Python package manager	pip	pip	Supported
Framework	react-vite	React + Vite	Golden stack
Framework	nextjs	Next.js	Golden stack
Backend framework	express	Express	Golden stack
Backend framework	fastapi	FastAPI	Golden stack
Backend framework	flask	Flask	Golden stack
Styling	tailwind	Tailwind CSS	Supported
UI	shadcn	shadcn/ui	Supported for compatible frameworks
Database	postgres	PostgreSQL	Project config + Docker option; native server not installed
Database	sqlite	SQLite	Supported
Database	mongodb	MongoDB	Project config + Docker option; native server not installed
ORM	prisma	Prisma	JS/TS
ODM	mongoose	Mongoose	MongoDB
ORM	sqlalchemy	SQLAlchemy	Python
Migration	alembic	Alembic	Python / SQLAlchemy
Validation	zod	Zod	JS/TS
Validation	pydantic	Pydantic	Python / FastAPI
Unit testing	vitest	Vitest	JS/TS
E2E testing	playwright	Playwright	Web projects
Testing	pytest	pytest	Python
Linting	eslint	ESLint	JS/TS
Formatting	prettier	Prettier	JS/TS
Quality	ruff	Ruff	Python
Infrastructure	docker	Docker	Detect + generate project files; runtime must exist
Infrastructure	docker-compose	Docker Compose	Detect + generate compose files
CI	github-actions	GitHub Actions	Generate workflow files

3.2 v1 golden stacks

Stack	Required integrations
Next.js full-stack	Node.js, pnpm/npm, Next.js, TypeScript, Tailwind, shadcn/ui optional, PostgreSQL or SQLite, Prisma, Zod, Vitest, Playwright, ESLint/Prettier
React SPA	Node.js, pnpm/npm, React + Vite, TypeScript, Tailwind, Zod, Vitest, Playwright, ESLint/Prettier
Express API	Node.js, pnpm/npm, Express, TypeScript, PostgreSQL or SQLite, Prisma, Zod, Vitest, ESLint/Prettier
FastAPI API	Python, uv/pip, FastAPI, Pydantic, PostgreSQL or SQLite, SQLAlchemy, Alembic, pytest, Ruff, Docker optional
Flask API	Python, uv/pip, Flask, PostgreSQL or SQLite, SQLAlchemy, Alembic, pytest, Ruff, Docker optional

Release discipline An integration is not considered supported merely because its package can be installed. It must satisfy the Definition of Done in Section 30.

4. User Experience and CLI Principles

4.1 Two interaction modes

Mode	Example	Audience
Interactive	\$ reposetup create	Beginners and exploratory use. Searchable prompts, defaults, explanations.
Non-interactive	\$ reposetup create my-app --framework nextjs --pm pnpm --add tailwind,prisma,zod	Experienced users, automation, CI, reproducibility.
Config-driven	\$ reposetup create --config reposetup.json	Canonical reproducible workflow.

4.2 Output style

Default output should be concise and operational. Detailed logs are available through --verbose. Color may improve readability but must never be the only signal; status symbols and text remain readable without ANSI color.

```
RepoSetup

Project: my-app
Framework: Next.js
Package manager: pnpm

Resolve
  OK 8 integrations compatible
  WARN PostgreSQL service is external; connection string required

Plan
  1 create project
  2 install dependencies
  3 configure Prisma
  4 create .env.example
  5 configure tests
  6 verify build

Continue? (Y/n)
```

4.3 Exit codes

Code	Meaning
0	Success
1	General execution failure
2	Invalid CLI usage or config
3	Compatibility/resolution failure
4	Prerequisite missing
5	Filesystem safety violation
6	Verification failed after setup
7	Network/package-manager failure
8	User cancelled

5. CLI Command Specification

The executable is `reposetup`. Commander.js owns command routing and flags. @inquirer/prompts is used only when information is missing and the process is running interactively.

Command	Purpose	Class
create	Create a new project from interactive choices, flags, config, or preset.	Mutating
add	Add one or more supported integrations to an existing project.	Mutating
remove	Remove a supported integration only when a safe reversible strategy exists.	Mutating / conservative
search	Search registry by name, keyword, category, or ecosystem.	Read-only
info	Show integration requirements, compatibility, options, and actions.	Read-only
list	List supported integrations/categories.	Read-only
stack	Detect and display current project stack.	Read-only
doctor	Run prerequisite and project-health checks.	Read-only by default
export	Write current detected RepoSetup config to a file.	Writes one requested file
import	Validate a RepoSetup config and optionally apply missing integrations.	Mutating only with --apply
preset	List/show local built-in presets.	Read-only
version	Print CLI and schema versions.	Read-only

5.1 reposetup create

```
reposetup create [project-name]
```

Options:

```
--framework <id>
--language <id>
--pm <npm|pnpm|bun|uv|pip>
--add <id,id,...>
```

```
--preset <id>
--config <path>
--dry-run
--yes
--verbose
--no-color
```

- If project name is omitted in an interactive terminal, prompt for it.
- If the target directory exists and is non-empty, fail unless the framework adapter explicitly supports safe initialization there.
- Always resolve and validate before filesystem mutation.
- Without `--yes`, show a summarized plan and require confirmation for mutating execution.
- With `--dry-run`, print the exact resolved operation plan and perform no mutation.
- After execution, run integration verification and a framework-level health check.

5.2 reposetup add

```
reposetup add <integration...>
reposetup add prisma zod
reposetup add --category testing
reposetup add prisma --dry-run
```

- Detect the current project before resolving additions.
- Never recreate the framework.
- Calculate only the delta between current state and desired state.
- If required choices are missing (e.g., Prisma requires a database), prompt in interactive mode or fail with a structured error in non-interactive mode.
- If an integration is already correctly configured, report it as satisfied rather than reinstalling blindly.

5.3 reposetup remove

Removal is intentionally conservative. Many framework generators and integrations modify user-owned files in ways that cannot be perfectly reversed.

- Remove only dependencies and files that RepoSetup can prove it owns or generated.
- Never delete user data, migrations, database contents, or application code.
- If safe reversal is unavailable, explain manual cleanup steps and exit without destructive mutation.
- `--force` must not bypass path-safety or ownership constraints; it may only skip non-destructive confirmation prompts.

5.4 Search and discovery

```
$ reposetup search orm

ID          NAME      CATEGORY  STATUS
prisma      Prisma    orm       stable
sqlalchemy  SQLAlchemy orm       stable
mongoose    Mongoose  odm       stable

$ reposetup info prisma
Prisma
Category: ORM
Requires: one database integration
Supports: nextjs, express, react-vite (client usage excluded), ...
Detects: package.json dependencies, prisma/schema.prisma
Actions: install, initialize, generate, verify
```

6. Configuration and Local Metadata

6.1 Canonical config file

The canonical portable configuration is `reposetup.json`. It is declarative, contains no executable code, and is safe to commit to version control. Schema versioning is independent from CLI versioning.

```
{
  "schemaVersion": 1,
  "project": {
    "name": "my-app"
  },
  "runtime": {
```

```

    "id": "node",
    "range": ">=22"
  },
  "packageManager": "pnpm",
  "framework": {
    "id": "nextjs",
    "options": {
      "typescript": true
    }
  },
  "integrations": [
    { "id": "tailwind" },
    { "id": "postgres", "options": { "mode": "external" } },
    { "id": "prisma" },
    { "id": "zod" },
    { "id": "vitest" }
  ]
}

```

6.2 Local metadata

A generated project may contain `.reposetup/state.json`. This file records what RepoSetup applied so future add/remove/doctor operations can distinguish managed artifacts from user-owned code.

```

{
  "schemaVersion": 1,
  "createdBy": "reposetup",
  "framework": "nextjs",
  "integrations": {
    "prisma": {
      "version": 1,
      "managedFiles": ["prisma/schema.prisma"],
      "managedEnvKeys": ["DATABASE_URL"]
    }
  }
}

```

Never store secrets `.reposetup.json` and `.reposetup/state.json` must never contain real passwords, API keys, auth tokens, or database credentials.

7. System Architecture

7.1 Monorepo

```

reposetup/
├── packages/
│   ├── cli/
│   ├── core/
│   ├── registry/
│   ├── integrations/
│   ├── executor/
│   ├── config/
│   └── testing/
├── presets/
├── examples/
├── docs/
├── scripts/
├── .github/workflows/
├── package.json
├── pnpm-workspace.yaml
├── tsconfig.base.json
├── CONTRIBUTING.md
├── SECURITY.md
└── LICENSE

```

7.2 Dependency direction

```
@repoSetup/cli
|
+--> @repoSetup/core
+--> @repoSetup/executor

@repoSetup/core
|
+--> @repoSetup/registry
+--> @repoSetup/config

@repoSetup/registry
|
+--> @repoSetup/integrations
```

Rules:

- core MUST NOT import CLI UI libraries
- registry MUST NOT execute shell commands
- integrations MUST NOT mutate the filesystem
- executor MUST NOT decide compatibility
- CLI MUST NOT contain package-specific setup logic

7.3 Package responsibilities

Package	Responsibility
@repoSetup/cli	Command routing, prompts, rendering, confirmations, exit codes.
@repoSetup/core	Resolve desired state, dependencies, conflicts, ordering, plan construction.
@repoSetup/registry	Load and validate integration definitions.
@repoSetup/integrations	Official integration definitions, detection rules, plan builders, verification hooks.
@repoSetup/executor	Filesystem adapter, safe process runner, operation execution, operation logs.
@repoSetup/config	Zod schemas, schema migrations, config serialization.
@repoSetup/testing	Fixtures, fake filesystem/process adapters, golden-stack helpers.

8. Core Domain Model

```
export type IntegrationCategory =
  | "framework"
  | "backend-framework"
  | "styling"
  | "ui"
  | "database"
  | "orm"
  | "odm"
  | "migration"
  | "validation"
  | "testing"
  | "linting"
  | "formatting"
  | "infrastructure"
  | "ci";

export interface SelectedIntegration {
  id: string;
  options?: Record<string, unknown>;
}

export interface RepoSetupConfig {
  schemaVersion: 1;
}
```

```

project: { name: string };
runtime?: { id: "node" | "python"; range?: string };
packageManager: "npm" | "pnpm" | "bun" | "uv" | "pip";
framework: { id: string; options?: Record<string, unknown> };
integrations: SelectedIntegration[];
}

```

8.1 Resolution context

```

interface ResolutionContext {
  config: RepoSetupConfig;
  platform: "darwin" | "win32" | "linux";
  detected?: DetectedProject;
  registry: IntegrationRegistry;
}

interface ResolutionResult {
  valid: boolean;
  desired: ResolvedStack;
  operations: InstallationOperation[];
  warnings: ResolutionMessage[];
  errors: ResolutionMessage[];
}

```

8.2 Detected project

```

interface DetectedProject {
  root: string;
  runtime?: DetectedRuntime;
  packageManager?: PackageManagerId;
  framework?: DetectedIntegration;
  integrations: DetectedIntegration[];
  files: Set<string>;
  envKeys: Set<string>;
  confidence: "high" | "medium" | "low";
}

```

9. Integration Registry

The registry is the product catalog. Each supported technology is implemented as an `IntegrationDefinition`. Integration files own their metadata, requirements, option schema, detection logic, operation builder, and verification logic.

```

interface IntegrationDefinition<TOptions = unknown> {
  id: string;
  name: string;
  category: IntegrationCategory;
  description: string;
  status: "experimental" | "stable" | "deprecated";
  docsUrl: string;

  optionSchema: ZodSchema<TOptions>;
  requires?: Requirement[];
  recommends?: Recommendation[];
  conflicts?: ConflictRule[];
  supports?: CompatibilityRule[];

  detect(ctx: DetectContext): Promise<DetectionResult>;
  buildPlan(ctx: PlanContext<TOptions>): InstallationOperation[];
  verify(ctx: VerifyContext<TOptions>): Promise<VerificationResult>;
}

```

9.1 Registry validation

- Integration IDs are globally unique and kebab-case.
- Every requirement reference resolves to a known integration or category.
- No dependency cycles are permitted.
- Every stable integration has official documentation URL and automated tests.
- Every option schema has defaults or explicit required values.
- Operation builders must not return arbitrary shell strings when structured command + args can be used.
- Deprecated integrations must provide an explanation and, when applicable, alternatives.

```
pnpm registry:validate
```

10. Compatibility and Dependency Resolver

10.1 Relationship types

Relationship	Meaning	Example
requires	Hard requirement; resolution fails without it.	Prisma requires one database selection.
recommends	Optional but useful companion.	React Hook Form could recommend Zod in future.
conflicts	Known unsupported combination.	An integration may conflict with a framework mode.
supports	Allowed compatibility.	Playwright supports selected web frameworks.
includes	Do not separately install included technology.	Next.js includes React.
alternative_to	Discovery hint only; not a conflict.	Prisma and Drizzle are alternative ORM choices.

10.2 Resolution pipeline

```

Raw Config / Add Request
  |
  v
Schema validation
  |
  v
Registry lookup
  |
  v
Normalize included integrations
  |
  v
Resolve requirements
  |
  v
Detect conflicts
  |
  v
Apply options/defaults
  |
  v
Build dependency graph
  |
  v
Topological order
  |
  v
Generate InstallationPlan

```

10.3 Resolution behavior

- Interactive mode may ask the user to satisfy an ambiguous requirement, such as choosing PostgreSQL or SQLite for Prisma.
- Non-interactive mode must never guess an unresolved hard requirement.
- Already-satisfied dependencies are not duplicated in the plan.
- Recommendations are displayed but never auto-installed unless a preset/config explicitly selected them.
- Framework-provided dependencies are marked as included and excluded from redundant install operations.

11. Installation Plan and Operations

Integrations return declarative operations. They never execute side effects directly. The planner may normalize or group operations before passing the plan to the executor.

```
type InstallationOperation =
  | EnsurePrerequisite
  | CreateProject
  | InstallPackages
  | RemovePackages
  | RunCommand
  | CreateFile
  | PatchJson
  | PatchText
  | EnsureEnvExampleKey
  | CreateDirectory
  | ShowInstruction
  | VerifyOperation;
```

11.1 Command operation

```
interface RunCommand {
  type: "run-command";
  executable: string;
  args: string[];
  cwd: string;
  env?: Record<string, string>;
  interactive: boolean;
  network: boolean;
  longRunning: boolean;
  allowFailure?: boolean;
}
```

11.2 File ownership

- Every file operation declares `create`, `merge`, or `replace` semantics.
- `replace` is forbidden on pre-existing user files unless the integration explicitly has a safe merger and the user confirms.
- Generated files are recorded in local RepoSetup state for future doctor/remove operations.
- Operation IDs must be stable enough for logs and debugging.

12. Package Manager and Runtime Adapters

12.1 JavaScript package manager adapter

```
interface JsPackageManagerAdapter {
  id: "npm" | "pnpm" | "bun";
  detect(root: string): Promise<DetectionResult>;
  install(deps: string[]): RunCommand;
  installDev(deps: string[]): RunCommand;
  remove(deps: string[]): RunCommand;
  exec(packageName: string, args: string[]): RunCommand;
  run(script: string, args?: string[]): RunCommand;
}
```

12.2 Python package manager adapter

```
interface PythonPackageManagerAdapter {
  id: "uv" | "pip";
  detect(root: string): Promise<DetectionResult>;
  install(deps: string[]): InstallationOperation[];
  installDev(deps: string[]): InstallationOperation[];
  run(moduleOrExecutable: string, args: string[]): RunCommand;
}
```

12.3 Runtime policy

Prerequisite	v1 behavior
Node.js	Detect executable and version. Do not install automatically.
Python	Detect executable and version. Account for `python`, `python3`, and Windows launcher where appropriate through a runtime adapter.
Git	Detect when a selected generator requires it.
Docker	Detect for Docker/Compose integration; generate project files but do not install Docker Desktop/Engine.
Database server	Prefer external connection or Docker Compose mode; do not install OS-level PostgreSQL/MongoDB packages automatically.

13. Project Creation

13.1 Framework adapter

```
interface FrameworkDefinition<TOptions> {
  id: string;
  name: string;
  optionSchema: ZodSchema<TOptions>;
  requiredRuntime: RuntimeRequirement;
  createProject(ctx: FrameworkCreateContext<TOptions>): InstallationOperation[];
  detect(ctx: DetectContext): Promise<DetectionResult>;
  verify(ctx: VerifyContext<TOptions>): Promise<VerificationResult>;
}
```

13.2 Create algorithm

1. Parse flags/config or collect interactive answers.
2. Validate project name and target path.
3. Detect runtime and package-manager prerequisites.
4. Resolve framework + selected integrations.
5. Generate full installation plan.
6. Present warnings and operation summary.
7. If dry-run, stop successfully here.
8. Create framework project using official generator or verified manual recipe.
9. Execute integration operations in dependency order.
10. Write `.reposeup/state.json` only after operations have successfully applied.
11. Run verification checks.
12. Print next-step commands and any manual secret/configuration requirements.

13.3 Project-name rules

- No path traversal (`..`).
- No shell metacharacter interpretation.
- No absolute path unless explicitly supported via `--dir` and normalized safely.
- No overwrite of non-empty directory by default.
- Framework-specific naming restrictions are validated before generator execution.

14. Existing Project Detection

Existing-project support is necessary for `add`, `stack`, `doctor`, `export`, and safe `remove`. Detection must be evidence-based and return confidence rather than assuming based on a single filename.

14.1 Evidence sources

Evidence	Examples
Manifest dependencies	package.json dependencies/devDependencies, pyproject.toml, requirements files
Lockfiles	pnpm-lock.yaml, package-lock.json, bun.lock/bun.lockb, uv.lock
Framework files	next.config.*, vite.config.*, manage.py, FastAPI app endpoints only when combined with dependencies
Config files	prisma/schema.prisma, eslint.config.*, playwright.config.*, docker-compose.*
RepoSetup state	.reposetup/state.json
Scripts	package.json scripts, pyproject tool sections

14.2 Confidence

Confidence	Rule
High	Multiple independent signals or RepoSetup state confirms integration.
Medium	One strong canonical signal.
Low	Heuristic only; display as possible but do not mutate based solely on it.

15. Add Integration Flow

```

$ cd existing-project
$ reposetup add prisma

Detected
  Framework: Next.js
  Package manager: pnpm
  Database: none

Prisma requires a database.
? Choose database PostgreSQL
? Database mode External connection

Plan
  install prisma @prisma/client
  create prisma/schema.prisma
  add DATABASE_URL to .env.example
  run Prisma generation

Continue? Yes

```

15.1 Delta planning

The planner compares detected current state against desired state. The resulting plan should contain only missing or incorrect pieces. Re-running `reposetup add prisma` should ideally become a no-op plus verification when Prisma is already healthy.

15.2 Existing configuration

- If a recognized config exists, use a merger rather than replacing it.
- If RepoSetup cannot safely merge the file, stop with an actionable warning.
- If a user file contains syntax errors, do not attempt blind patching.
- Back up a file before non-trivial merge operations if the merger cannot guarantee atomicity.

16. Remove and Rollback Policy

Removal is not the inverse of installation in many ecosystems. A package can add migrations, source files, environment variables, or user customizations after installation. v1 removal therefore follows an ownership model.

16.1 Safe removal

- Remove dependency entries owned by the integration when no other selected integration requires them.
- Remove files only when `.reposetup/state.json` marks them as generated and their content fingerprint still matches the generated version or a recognized merge-safe state.
- Do not delete database migrations, application routes, schemas with user changes, uploaded assets, or data.
- Do not attempt to rollback framework generators.
- If removal is unsafe, print exact remaining manual cleanup items.

16.2 Failure during create/add

The executor maintains an operation journal. For operations explicitly marked reversible, it may undo them in reverse order after failure. Otherwise it reports the partially-applied state and leaves diagnostic logs. v1 must prefer transparency over pretending a full rollback succeeded.

17. Search, Info, Stack, Doctor, Export and Import

17.1 stack

```
$ reposetup stack

Runtime
Node.js detected

Framework
Next.js high confidence

Integrations
Tailwind CSS high
Prisma high
PostgreSQL medium (DATABASE_URL + Prisma provider)
Vitest high

Package manager
pnpm
```

17.2 doctor

Doctor is one of the most valuable terminal-first features. It runs non-destructive checks by default and should be useful even for projects not originally created by RepoSetup.

```
$ reposetup doctor

Prerequisites
OK Node.js satisfies required range
OK pnpm available

Project
OK Next.js detected
WARN DATABASE_URL missing from local environment
FAIL Prisma client appears not generated

Suggested action
reposetup doctor --fix-safe
```

- ``doctor`` performs no mutation.
- ``doctor --fix-safe`` may apply only operations marked safe and non-destructive, such as regenerating derived clients or adding a missing example env key.
- Anything requiring secrets, destructive migration, or ambiguous user intent remains manual.

17.3 Export/import

```

reposest export --output reposest.json
reposest import reposest.json
reposest import reposest.json --apply --dry-run

```

Export serializes the detected supported portion of the stack. Unsupported dependencies remain outside the config and may be listed as informational extras.

18. Presets

v1 presets are local, version-controlled JSON files shipped with the repository or installed package. They are declarative config templates and never contain arbitrary commands.

```

presets/
├── next-fullstack.json
├── react-spa.json
├── express-api.json
├── fastapi-api.json
└── flask-api.json

reposest preset list
reposest preset show next-fullstack
reposest create my-app --preset next-fullstack

```

18.1 Preset rules

- Preset schema must validate with the same config schema.
- Presets may contain defaults but can be overridden through CLI options.
- Presets cannot include shell scripts, URLs to executable code, or secrets.
- Built-in presets must be integration-tested as golden stacks.

19. Filesystem and Configuration Modification

19.1 Atomicity

- Write new file content to a temporary file in the same filesystem, then rename atomically where supported.
- Create parent directories only inside the normalized project root.
- Reject any operation whose resolved path escapes the project root unless explicitly marked as read-only prerequisite detection.
- For JSON, parse -> modify structured data -> serialize; never regex-edit JSON.
- For supported JS/TS config files, prefer dedicated adapters or AST transforms when non-trivial merge is needed.
- For simple text configs, use sentinel-aware patching and verify the result.

19.2 Merge strategy hierarchy

1. Use an official CLI/generator when it safely performs the configuration.
2. Use structured AST/JSON/YAML/TOML merger.
3. Use template creation when the file does not exist.
4. Use narrow text patch only when structure is stable and tests cover it.
5. Otherwise stop and give manual instructions.

20. Environment Variables and Secrets

Integrations declare environment variable requirements. RepoSetup may create or update ``.env.example``; it must not collect or upload secrets.

```

interface EnvRequirement {
  name: string;
  required: boolean;
  secret: boolean;
  scope: "server" | "client" | "build";
  example?: string;
}

```

```
description: string;
}
```

20.1 Behavior

- Add missing keys to ``.env.example`` with placeholders.
- Do not overwrite an existing local ``.env`` value.
- Do not print secret values in logs.
- Do not include secret values in operation journals.
- If an integration requires a secret before verification can pass, mark verification as blocked and explain the expected key.

21. Security and Safe Command Execution

21.1 Process execution

```
// Preferred
execa(executable, args, { cwd, shell: false });

// Avoid
exec(`${executable} ${userInput}`);
```

- Use `executable + args` arrays, not interpolated shell strings.
- Use `shell: false` by default.
- If an official workflow truly requires shell syntax, mark the operation explicitly and constrain all variable inputs.
- Validate project names, paths, integration options, ports, and identifiers with Zod before use.
- Never execute commands supplied by remote config/presets.
- Never run ``curl | sh`` style installation as part of normal project setup.
- No sudo/admin elevation in v1.
- All external command invocations are logged with secret redaction.

21.2 Supply-chain boundary

RepoSetup necessarily invokes official ecosystem package managers. That does not make every package trustworthy. Official integrations must pin package names and use verified official installation sources. Community contribution review should check package ownership, official repository/docs, and typosquatting risk.

21.3 Path safety

- Normalize paths before use.
- Reject ``.`` traversal after normalization.
- Reject symlink-based writes that escape project root where feasible.
- Never recursively delete project directories as part of an integration.
- Destructive database commands are outside automatic v1 setup.

22. Error Model and Recovery

```
type RepoSetupErrorCode =
  | "INVALID_CONFIG"
  | "UNKNOWN_INTEGRATION"
  | "MISSING_REQUIREMENT"
  | "INTEGRATION_CONFLICT"
  | "PREREQUISITE_MISSING"
  | "UNSUPPORTED_PLATFORM"
  | "UNSAFE_PATH"
  | "FILE_CONFLICT"
  | "PACKAGE_MANAGER_FAILED"
  | "COMMAND_FAILED"
  | "VERIFICATION_FAILED"
  | "USER_CANCELLED";
```

22.1 Error UX

```
Cannot add Prisma.
```

```
Reason
  Prisma requires a database integration.

Choose one interactively:
  reposetup add prisma

Or specify it explicitly:
  reposetup add postgres prisma
```

22.2 Partial application

On failure, report completed, failed, skipped, and rolled-back operations. Always preserve a machine-readable journal for diagnostics. Do not hide partial state behind a generic error.

23. Logging and Diagnostics

23.1 User-facing levels

Mode	Output
Default	Progress, warnings, concise failures, final summary.
--verbose	Commands, operation IDs, timing, detection evidence, file paths.
--json	Machine-readable result for automation where supported.
--no-color	Disable ANSI formatting.

23.2 Diagnostic file

Failed mutating commands may write ``.reposetup/logs/<timestamp>.jsonl``. Logs should be excluded from version control and redact secret-bearing environment variable values.

24. Testing Strategy

24.1 Test layers

Layer	Purpose	Examples
Unit	Pure domain behavior	Zod schema, resolver requirements, conflict rules, topological sort.
Integration definition	Plan generation/detection	Prisma plan for PostgreSQL; Tailwind detection.
Executor	Side-effect adapters	Fake filesystem, command failure, atomic writes, path safety.
CLI	Command UX and flags	--help, invalid args, non-interactive failure, exit codes.
Golden stack E2E	Real temporary project setup	Next full-stack, React SPA, Express API, FastAPI, Flask.
Cross-platform smoke	Platform differences	Linux in CI mandatory; Windows/macOS runners for critical flows as budget permits.

24.2 Snapshot policy

Snapshot tests are appropriate for normalized installation plans and generated config files, but tests must also assert key semantics. Avoid giant snapshots that become blindly updated whenever a dependency changes.

24.3 Network isolation

Most unit/integration tests should not require network. E2E installation jobs may access package registries and should be separated, cached carefully, and allowed to fail with actionable evidence when external registries are unavailable.

25. Golden Stack Certification

A golden stack is a complete, high-value combination that RepoSetup guarantees and continuously tests. Golden stacks are more meaningful than an abstract count of supported packages.

Golden stack	Required verification
Next.js full-stack	Creation succeeds, package install succeeds, config generated, typecheck/build succeeds, Prisma generation succeeds when DB choice allows offline verification, Vitest config loads, Playwright config loads.
React SPA	Vite dev/build config valid, Tailwind configured, Vitest passes starter smoke test, Playwright config valid.
Express API	TypeScript compiles, server imports, Prisma/Zod configuration valid, Vitest starter test passes.
FastAPI API	Application imports, uv/pip environment valid, SQLAlchemy/Alembic configured, pytest starter test passes, Ruff check passes.
Flask API	Application imports, SQLAlchemy/Alembic configured, pytest starter test passes, Ruff check passes.

25.1 Certification state

State	Meaning
stable	Golden path passes CI and current integration metadata is verified.
experimental	Usable but not yet fully certified across target combinations/platforms.
needs-review	Known upstream changes or failing scheduled health test.
deprecated	Maintained only for migration/history; new projects should not select by default.

26. CI/CD and Release Engineering

26.1 GitHub Actions workflows

```
.github/workflows/
├── ci.yml
├── registry-validate.yml
├── golden-stacks.yml
├── cross-platform.yml
├── release.yml
└── scheduled-integration-health.yml
```

26.2 CI gates

- Install dependencies with frozen lockfile.
- Lint/format check.
- Typecheck every package.
- Run unit and integration tests.
- Run registry validation.
- Build CLI package.
- Run safe CLI smoke test (`reposetup --help`, `list`, dry-run fixture).

- For release branches/tags, run golden stack jobs before publication.

26.3 Versioning

- Use semantic versioning for CLI packages.
- Maintain independent `schemaVersion` for `reposetup.json`.
- Provide config migration functions for supported schema versions.
- Never silently reinterpret an old config with incompatible semantics.

26.4 Publishing

Publish a package that exposes the `reposetup` binary. Release artifacts should include a changelog and explicit integration changes. Standalone native binaries may be explored later; they are not required for v1.

27. Open-Source Contribution Model

27.1 Integration contribution

```
packages/integrations/src/prisma/
├── definition.ts
├── options.ts
├── detect.ts
├── plan.ts
├── verify.ts
├── fixtures/
└── prisma.test.ts
```

27.2 Pull-request checklist

- Official technology name and unambiguous integration ID.
- Official documentation and official package names verified.
- Supported versions or version policy documented.
- Options schema provided.
- Requirements/conflicts/recommendations defined.
- Detection behavior tested.
- Installation plan tested for each supported package manager.
- File modifications are safe and covered by fixtures.
- Verification hook implemented.
- At least one real integration test added or updated.
- Documentation page added.

27.3 Maintainer review

Maintainers should reject integrations that primarily execute opaque downloaded scripts, cannot be validated safely, or have unclear package ownership. The registry is curated, not an unrestricted index of every package ecosystem.

28. Documentation Requirements

28.1 User docs

- Getting started and installation.
- Interactive create tutorial.
- Non-interactive examples.
- Adding integrations.
- Doctor and troubleshooting.
- Config file reference.
- Preset reference.
- Supported integration matrix.
- Security model and dry-run explanation.

28.2 Contributor docs

- Monorepo architecture.
- How the resolver works.
- How to create an integration.
- How to write filesystem mergers.
- How to add golden-stack tests.
- How to research and verify official setup commands.
- Release process.

28.3 Integration documentation template

```
# Prisma

Category: ORM
Status: stable
Official docs: <url>

## Requires
- one supported database

## Supported package managers
- npm
- pnpm
- bun (only if verified)

## RepoSetup actions
- install packages
- create/merge Prisma configuration
- add DATABASE_URL example
- generate client

## Detection
...

## Verification
...

## Known limitations
...
```

29. Implementation Roadmap

Phase	Focus	Exit condition
Phase 0	Repository foundation	pnpm monorepo, TypeScript config, lint/format, Vitest, CI skeleton.
Phase 1	Core schemas	RepoSetupConfig, Zod schemas, errors, schema versioning.
Phase 2	Registry	IntegrationDefinition, registry loader, validator, categories.
Phase 3	Resolver	Requirements, conflicts, includes, graph ordering, deterministic results.
Phase 4	Operation model	InstallationOperation union, planner normalization, dry-run rendering.
Phase 5	CLI shell	Commander commands, Inquirer prompts, output renderer, exit codes.
Phase 6	Executor safety	execa runner, filesystem adapter, atomic writes, operation journal.
Phase 7	First JS path	Node detect, pnpm/npm, Next.js, Tailwind, SQLite, Prisma, Zod.
Phase 8	React/Express paths	React + Vite, Express, Vitest, ESLint, Prettier,

Phase	Focus	Exit condition
		Playwright.
Phase 9	Existing project	Detection engine, stack, add, export, doctor.
Phase 10	Python path	Python detect, uv/pip, FastAPI, Flask, Pydantic, SQLAlchemy, Alembic, pytest, Ruff.
Phase 11	Infra	PostgreSQL/MongoDB project configuration, Docker/Compose, GitHub Actions.
Phase 12	Removal	Safe ownership-based remove for integrations with reversible implementation.
Phase 13	Golden certification	Five golden stacks, cross-platform smoke, docs.
Phase 14	v1 release	Package publish, changelog, security doc, stable schema.

29.1 Implementation order rule

Do not start by implementing every integration Build the resolver + plan + executor architecture first, then certify one complete stack end-to-end. Add breadth only after the architecture proves itself.

30. Definition of Done

30.1 Integration Definition of Done

- Metadata and official docs recorded.
- Option schema validates.
- Requirements/conflicts/support rules defined.
- Detection implemented with confidence.
- Plan generation is deterministic.
- Package-manager variants are tested.
- File changes use safe create/merge semantics.
- Environment variable requirements are declarative.
- Dry-run output is meaningful.
- Execution path has unit/integration coverage.
- Verification hook exists.
- Known limitations documented.
- At least one compatible golden-stack or dedicated E2E scenario exercises it.

30.2 v1 Release Definition of Done

- ``create``, ``add``, ``search``, ``info``, ``list``, ``stack``, ``doctor``, ``export``, ``import``, and local ``preset`` flows are stable.
- ``remove`` exists for integrations with safe reversible strategies and refuses unsafe removals clearly.
- Config schema v1 is documented and migration infrastructure exists.
- Five golden stacks pass release CI.
- Linux is fully supported; Windows and macOS critical flows have documented smoke coverage.
- No v1 command requires a website or remote RepoSetup account.
- No arbitrary remote commands are executed.
- No default path requires `sudo/admin`.
- CLI help, README, contributor guide, security model, and integration author guide are complete.
- Package can be installed and invoked through the chosen npm distribution workflow.

Appendix A. v1 Integration Registry

ID	Category	Name	Requires	v1 role	Target confidence
node	runtime	Node.js	-	Prerequisite only; detect version	High
python	runtime	Python	-	Prerequisite only; detect version	High
npm	package-manager	npm	node	JS dependency operations	High
pnpm	package-manager	pnpm	node	Preferred JS adapter	High
bun	package-manager	Bun	-	Package-manager use only where verified	Medium
uv	package-manager	uv	python	Preferred Python adapter	High
pip	package-manager	pip	python	Python dependency adapter	High
nextjs	framework	Next.js	node + JS PM	Framework create/detect/verify	High
react-vite	framework	React + Vite	node + JS PM	Framework create/detect/verify	High
express	backend-framework	Express	node + JS PM	API project recipe	High
fastapi	backend-framework	FastAPI	python + Python PM	API project recipe	High
flask	backend-framework	Flask	python + Python PM	API project recipe	High
tailwind	styling	Tailwind CSS	compatible framework	Framework-specific recipes	High
shadcn	ui	shadcn/ui	compatible React framework	Official CLI integration	Medium
postgres	database	PostgreSQL	-	External or Docker mode	High
sqlite	database	SQLite	-	File database	High
mongodb	database	MongoDB	-	External or Docker mode	Medium
prisma	orm	Prisma	one DB + JS PM	Schema/client configuration	High
mongoose	odm	Mongoose	MongoDB + JS PM	MongoDB model layer	Medium
sqlalchemy	orm	SQLAlchemy	Python PM	Python ORM	High
alembic	migration	Alembic	SQLAlchemy	Migration setup	High
zod	validation	Zod	JS PM	Validation dependency	High
pydantic	validation	Pydantic	Python PM	Validation/models	High
vitest	testing	Vitest	JS project	Unit testing	High
playwright	testing	Playwright	Web project	E2E + browser install	High
pytest	testing	pytest	Python project	Testing	High
eslint	linting	ESLint	JS project	Lint configuration	High
prettier	formatting	Prettier	JS project	Formatting	High
ruff	quality	Ruff	Python project	Lint/format	High
docker	infrastructure	Docker	Docker runtime present	Generate Docker project files	Medium
docker-compose	infrastructure	Docker Compose	Docker present	Local services orchestration	Medium
github-actions	ci	GitHub Actions	Git repo optional	Generate workflows	High

Appendix B. Example Configurations

B.1 Next.js full-stack

```
{
  "schemaVersion": 1,
  "project": { "name": "web-app" },
  "runtime": { "id": "node" },
  "packageManager": "pnpm",
  "framework": {
    "id": "nextjs",
    "options": { "typescript": true }
  },
  "integrations": [
    { "id": "tailwind" },
    { "id": "postgres", "options": { "mode": "external" } },
    { "id": "prisma" },
    { "id": "zod" },
    { "id": "vitest" },
    { "id": "playwright" },
    { "id": "eslint" },
    { "id": "prettier" }
  ]
}
```

B.2 FastAPI API

```
{
  "schemaVersion": 1,
  "project": { "name": "api" },
  "runtime": { "id": "python" },
  "packageManager": "uv",
  "framework": { "id": "fastapi" },
  "integrations": [
    { "id": "pydantic" },
    { "id": "postgres", "options": { "mode": "docker" } },
    { "id": "sqlalchemy" },
    { "id": "alembic" },
    { "id": "pytest" },
    { "id": "ruff" },
    { "id": "docker" },
    { "id": "docker-compose" }
  ]
}
```

Appendix C. Cursor Implementation Instructions

Use this document as source of truth When implementation details are ambiguous, preserve the architecture boundaries and safety model rather than introducing shortcuts. Do not add website/backend scope to v1.

1. Create the monorepo and shared TypeScript configuration.
2. Implement config schemas and error types before CLI UI.
3. Implement registry and registry validation.
4. Implement pure resolver logic with unit tests.
5. Implement InstallationOperation and normalized plan types.
6. Implement `reposetup plan` internally or dry-run rendering before real execution.
7. Implement executor behind interfaces using fake filesystem/process adapters in tests.
8. Implement Node/runtime and npm/pnpm prerequisite detection.
9. Implement one golden stack: Next.js + Tailwind + SQLite + Prisma + Zod.
10. Only after that path passes E2E tests, add React/Vite and Express.

11. Implement existing-project detector, `stack`, and `add`.
12. Implement Python adapter and FastAPI path.
13. Add doctor/export/import and remaining v1 integrations.
14. Implement safe remove only for integrations that can prove ownership.
15. Run golden stack certification, cross-platform smoke tests, and release gates.

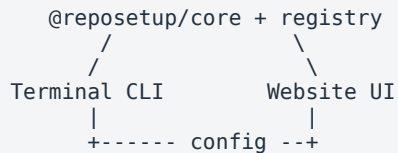
C.1 Forbidden shortcuts

- Do not put `if (integration === "prisma")` logic throughout CLI components.
- Do not let integration definitions call `execa/fs` directly.
- Do not use shell interpolation with raw user input.
- Do not replace pre-existing user configuration files blindly.
- Do not auto-select missing required dependencies in non-interactive mode.
- Do not add a database/server/backend for RepoSetup itself in v1.
- Do not implement an unrestricted remote plugin system.
- Do not use AI to synthesize installation commands at runtime.

Appendix D. Future Website Boundary

The website is intentionally deferred. The terminal product must not be architected as a temporary prototype that depends on a future server. A future website should consume the same registry metadata and config schema rather than duplicating installation logic.

Future only:



The website may later provide discovery, visual composition, shared presets, and documentation. The CLI remains capable of executing local configs offline.

D.1 What must remain reusable

- Integration metadata and categories.
- Config schema.
- Resolver output and warnings.
- Installation-plan preview model.
- Preset format.
- Compatibility information.

Implementation Starting Point

The first milestone to give Cursor

Milestone 1

Goal:

Given a validated RepoSetupConfig, produce a deterministic InstallationPlan.
No real package installation yet.

Required packages:

- @reposetup/config
- @reposetup/registry
- @reposetup/core
- @reposetup/integrations (fixtures only at first)

Acceptance criteria:

1. Config parses with Zod.
2. Unknown integration fails clearly.
3. Hard requirements are resolved or rejected.
4. Conflicts are rejected.
5. Includes are de-duplicated.
6. Dependency graph is topologically sorted.
7. Same input yields stable operation ordering.
8. Dry-run plan can be rendered as text.
9. Unit tests pass.
10. No filesystem or process execution exists in core.

Recommended first real path After Milestone 1, implement Next.js + pnpm + Tailwind + SQLite + Prisma + Zod as the first end-to-end certified stack. This gives RepoSetup a narrow but meaningful working product before expanding the catalog.